Data Sources : Structured and unstructured data

Sources :

Claim and Website Database

Rest API & CRM Database

Claim, Finance, Product, Sales Department:

Data Source Description:

Data is primarily sourced from the backend database managed in PostgreSQL in Azure SQL.

Data Capture Method:

Real-time data capturing is facilitated through Debezium CDC (Change Data Capture), ensuring up-to-date financial information through Apache Kafka

Additional Tools:

Utilization of spreadsheets for data analysis and reporting.

Marketing Department:

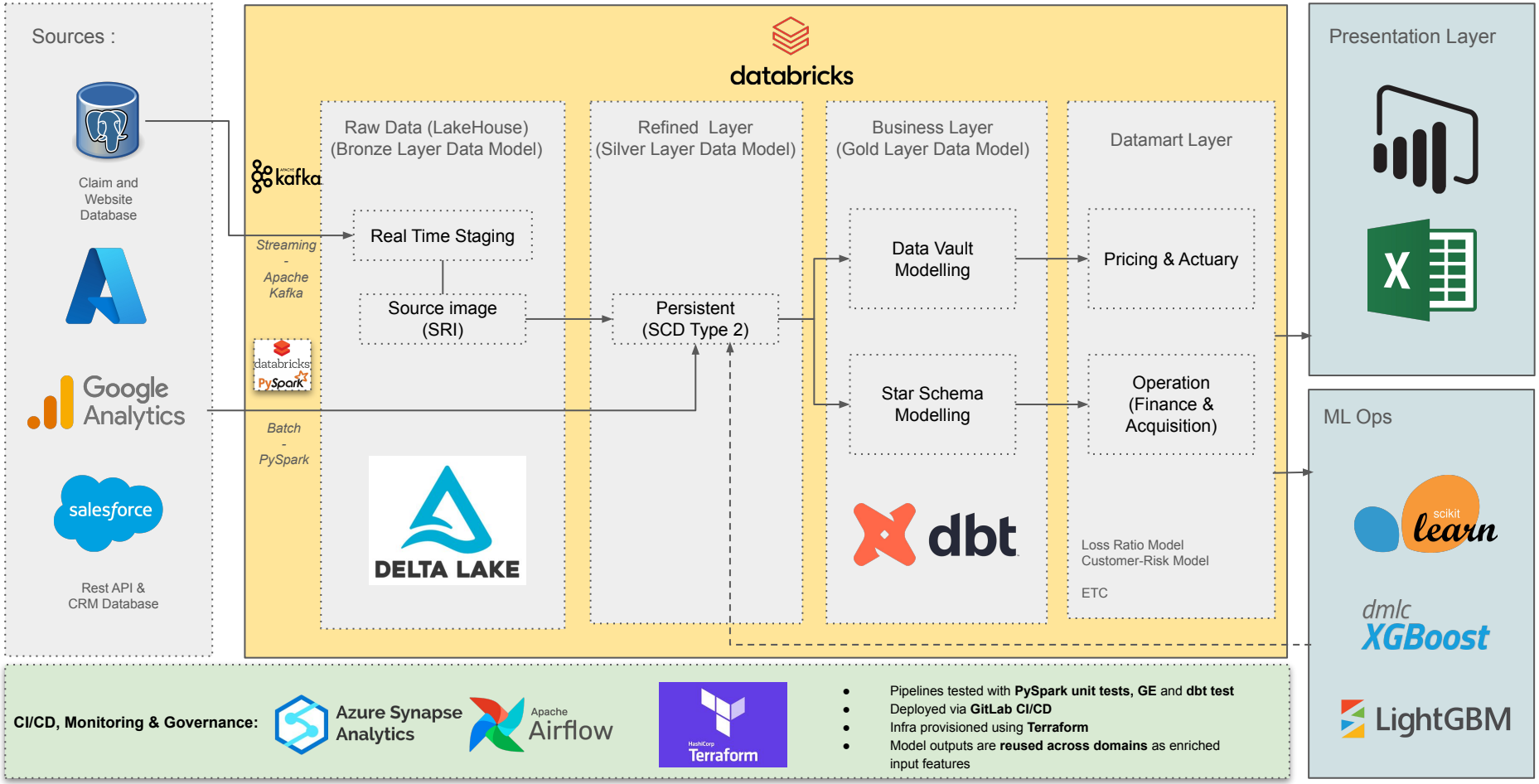
Data Source Description:

Data is sourced from Google Analytics 4 (GA4) and Sales Force like Quotations and Orders

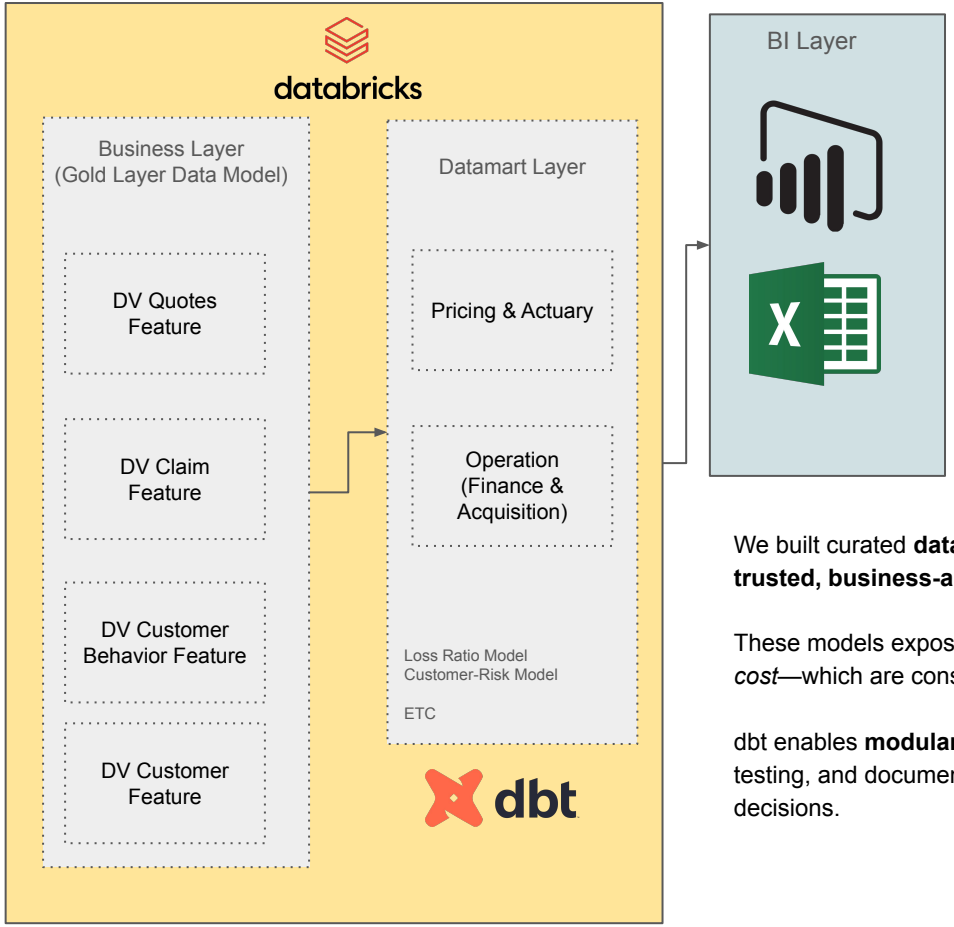
Purpose:

This data is used for tracking marketing campaigns, customer engagement, and website analytics.

High-Level Architecture Overview - Lambda Architecture



Datamart-Driven 360 Business Intelligence - *Extended Hybrid Architecture for Centralised Analytics*



Trusted KPI Delivery for Business Teams

📈 ***Pricing & Actuary***

- `avg_premium_per_risk_level`
- `loss_ratio_by_product`
- `price_sensitivity_model_output`

🧾 ***Operation (Finance & Acquisition)***

- `claim_payout_forecast`
- `customer_lifetime_value`
- `acquisition_cost_per_channel`

We built curated **datamarts** in Databricks using **dbt**, transforming **DV-based features** into **trusted, business-aligned models** for teams like Finance and Actuarial.

These models expose **critical KPIs**—including *loss ratio, predicted premium, and acquisition cost*—which are consumed directly in **Power BI** and **Excel**.

dbt enables **modular, testable, and explainable pipelines** by enforcing version control, model testing, and documentation. This gives stakeholders **confidence in the data** they use to drive decisions.

Data Mesh-Driven OpenAI Integration – Scalable LLM Architecture for Customer Support and Lead Generation

LLMs in has two phases:

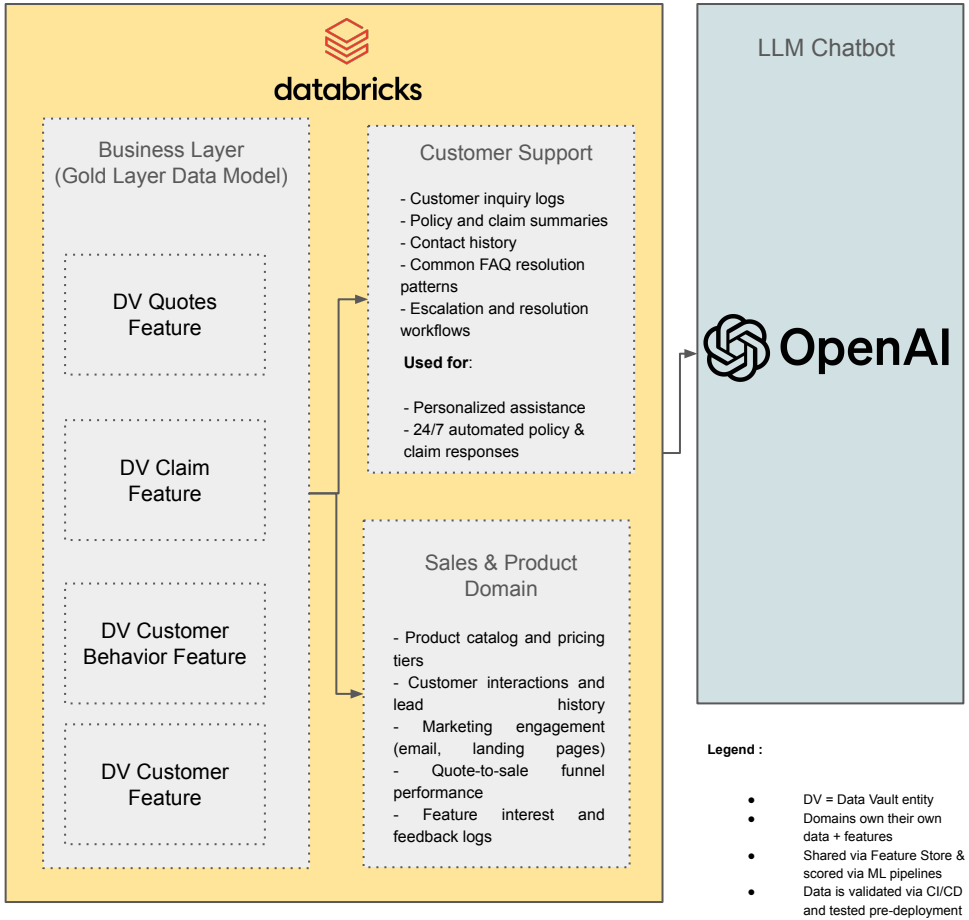
First, curated data products from key domains (**Quotes, Claims, Customers**) are modeled using **DBT + Delta Lake, validated via Great Expectations, and versioned via CI/CD.**

Then, LLM endpoints (via OpenAI API) are exposed via a secure internal layer. Prompts are enriched using metadata and contextual embeddings retrieved from the **Databricks gold layer**.

LLM Use cases	Domain	Features From	Output
GPT Chatbot for Support	Customer Support ops and FAQ	DV Quotes, DV Claims, DV Customer Behavior	Contextual response, FAQ
LightGBM	Sales & Product Domain	DV Quotes, Product Metadata	Lead classification, CTA

Notes:

Each DV-based domain enforces a schema contract for shared features. Changes are tracked and tested in CI/CD, ensuring downstream models are protected from unexpected schema shifts



Code Paradigms & Architecture Patterns in Our Data Platform

We apply a mix of paradigms depending on the context — functional patterns in PySpark, declarative logic in DBT, and TDD for reliability. Architecturally, we follow a **layered Lakehouse**, apply **CQRS for scoring vs analytics**, and treat each domain as a **data product** in line with **Data Mesh**.

Paradigm	Description
Functional Programming	Used in PySpark transformations to keep logic stateless and composable
Object-Oriented Design	Modular data transformers, testable classes (e.g., QuoteRiskTransformer)
Declarative Programming	Used in DBT and Terraform to define models/infrastructure cleanly
Event-Driven Design	Trigger ML scoring pipelines via Kafka events (claims, quotes)
Test-Driven Development (TDD)	We write tests first in PyTest + Chispa, then implement logic

Pattern	Usage
Layered Architecture	Bronze → Silver → Gold Delta Lake model in Databricks
Hexagonal Architecture	ETL core logic separated from storage, APIs, or output adapters
CQRS	Scoring jobs (write) separated from reporting (read/BI)
Event Sourcing	All data changes stored as Kafka CDC events (Debezium, GA4, etc.)
Data Mesh	Domains own their DV entities (Pricing, Fraud, Predictive) with contracts

Orchestration and Terraform as Infrastructure as a code Effort

Manage, Orchestration, and Monitor :

Azure Synapse Analytics

Apache Airflow

HashiCorp Terraform

Workflow Orchestration:

Automated scheduling of data pipelines using tools like Apache Airflow. Coordination of both real-time (Kafka streaming) and batch (PySpark) processing.

Monitoring:

End-to-end monitoring of data flows and pipeline performance. Alerts and logging to ensure data quality and reliability.

Benefits:
Increased efficiency and reduced manual intervention.
Timely error detection and resolution.

Provisioning Azure Resources:

Automated deployment of Azure services including Azure Databricks clusters, Azure Synapse Analytics jobs, Blob Storage accounts, and Azure Data Factory pipelines.

Version Control & Reproducibility:

Infrastructure as Code (IaC) with Terraform enables versioning and rollback capabilities.

Integration:

Smooth integration with orchestration tools ensures resources are provisioned on-demand. Unified management of the entire Azure-base data architecture.